PROJECT REPORT FOR

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

AI-BASED 8-PUZZLE SOLVER

A CLASSICAL SEARCH-BASED APPROACH TO INTELLIGENT PROBLEM SOLVING

A PROJECT REPORT

Submitted by

NAME: _____

REGISTER NUMBER: _____

Department of Computer Science and Engineering

Rajalakshmi Engineering College

October 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “AI-BASED 8-PUZZLE SOLVER” is the bonafide work of the student(s) named below, carried out as part of the Foundations of Artificial Intelligence course under the supervision of the faculty in-charge.

NAME: _____

REGISTER NUMBER: _____

SIGNATURE OF THE FACULTY IN-CHARGE

Faculty Name: _____

Designation: _____

Department of Computer Science and Engineering

Submitted for the Practical Examination held on: _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	6
3	PROPOSED SYSTEM	9
3.1	ARCHITECTURE DIAGRAM	10
4	MODULES DESCRIPTION	11
4.1–4.6	MODULE DETAILS	11–12
5	IMPLEMENTATION AND RESULTS	13
5.1	EXPERIMENTAL SETUP	13
5.2	RESULTS AND DISCUSSION	14–16
6	CONCLUSION AND FUTURE WORK	17
7	REFERENCES	18

ABSTRACT

The 8-puzzle is a classic artificial intelligence problem consisting of a 3 × 3 board containing eight numbered tiles and one blank space. The objective is to transform a given initial arrangement into a specified goal arrangement by sliding one adjacent tile into the blank space. Although the rules are simple, selecting a sequence of moves efficiently requires state-space search and a suitable strategy for exploring alternatives.

This project presents an AI-Based 8-Puzzle Solver that models each board arrangement as a state and each legal tile movement as an action. The solver can use Breadth-First Search (BFS) for finding a shortest solution on small search spaces and A* search for more efficient informed search. In A*, the evaluation function f(n) = g(n) + h(n) combines the cost already incurred, g(n), with a heuristic estimate, h(n), of the remaining distance. The Manhattan-distance heuristic estimates how far each tile is from its goal position.

The proposed application accepts an initial board, checks whether the arrangement is solvable, searches for a path to the goal, and displays the sequence of intermediate boards and the total number of moves. The approach demonstrates important FOAI concepts including state-space representation, operators, goal testing, uninformed search, heuristic search, and algorithm evaluation. The report describes the design and implementation plan; actual performance values should be filled in after running the program on the target system.

CHAPTER 1

INTRODUCTION

Artificial Intelligence (AI) studies methods that enable computer systems to solve problems, reason about alternatives, and select actions toward a goal. Search is one of the foundational techniques in AI. A search problem typically contains an initial state, a set of possible actions, a transition model, a goal test, and a path-cost function.

The 8-puzzle is a widely used example for understanding search algorithms. It consists of a 3 × 3 grid with eight numbered tiles and one empty cell. A move is made by sliding a tile adjacent to the blank into the blank cell. The task is to reach a target arrangement, commonly 1 2 3 / 4 5 6 / 7 8 blank.

The challenge is not to make any legal move, but to find a sequence that reaches the goal without wasting unnecessary steps. Uninformed search methods explore states without estimating their distance to the goal, whereas informed methods use heuristic knowledge to prioritize promising states. This project applies these ideas to construct a clear and explainable 8-puzzle solver.

1.1 PROBLEM STATEMENT

Given a valid initial 3 × 3 board configuration, determine whether the goal configuration is reachable and, if it is, find a sequence of legal moves that transforms the initial state into the goal state. The solver should avoid revisiting already explored states and should report the solution path and move count.

1.2 OBJECTIVES

• Represent the puzzle as a finite state-space search problem.

• Generate only legal actions based on the position of the blank cell.

• Check solvability before starting an expensive search.

• Implement A* search using Manhattan distance as a heuristic.

• Reconstruct and display the sequence of board states leading to the goal.

• Evaluate solution length, explored states, and execution time across test cases.

1.3 SCOPE

The project focuses on a single-agent, deterministic puzzle with a fixed 3 × 3 board. It is intended for learning and demonstration, rather than as a general-purpose planning system. The design can later be extended to compare algorithms, animate moves, or support larger sliding puzzles.

CHAPTER 2

LITERATURE REVIEW

Research and teaching in classical AI have established state-space search as a basic problem-solving framework. The 8-puzzle is frequently used to compare search strategies because its state transitions are simple to define, while the number of possible arrangements makes search efficiency important.

Approach / Source	Main idea	Limitation	Relevance to this project
Breadth-First Search (BFS)	Explores states level by level and finds a shortest path when each move has equal cost.	Can consume substantial memory as the frontier grows.	Provides a baseline for comparing solution length and explored states.
Depth-First Search (DFS)	Explores one branch deeply before backtracking.	Does not generally guarantee the shortest solution and can spend time on poor branches.	Useful for explaining uninformed search and its trade-offs.
A* Search (Hart, Nilsson & Raphael, 1968)	Ranks nodes using f(n) = g(n) + h(n).	Memory use can still be high for difficult instances.	Main informed-search method in the proposed solver.
Manhattan Distance	Sums each numbered tile's horizontal and vertical distance from its goal position.	Does not account for all interactions among tiles.	An admissible heuristic for standard 8-puzzle moves.
Russell & Norvig (2021)	Presents problem formulation, search, and heuristic evaluation in modern AI.	A broad textbook rather than a puzzle-specific implementation.	Provides the conceptual basis for the state-space design.

The literature and standard AI texts suggest that search algorithms should be compared not only by whether they find a solution, but also by completeness, optimality, memory consumption, and number of expanded states. BFS is complete and optimal for equal-cost moves but may use large amounts of memory. A* can reduce unnecessary exploration when its heuristic is informative. Manhattan distance never overestimates the number of moves needed in the standard 8-puzzle, so A* using this heuristic is optimal under the usual conditions.

The contribution of this student project is an integrated, understandable implementation that connects puzzle representation, solvability checking, heuristic evaluation, path reconstruction, and a visible solution sequence in one workflow.

CHAPTER 3

PROPOSED SYSTEM

The proposed AI-Based 8-Puzzle Solver accepts an initial board from the user and uses a fixed goal board. Before search begins, it validates the input and checks solvability using inversion parity. For a 3 × 3 puzzle, the initial state is solvable to the standard goal only when the inversion count of the numbered tiles is even.

The board is encoded as a tuple of nine values, with 0 representing the blank. The state generator locates the blank and creates a successor state for every legal move: up, down, left, or right, when that direction remains inside the grid. A* maintains a priority queue ordered by f(n) = g(n) + h(n), where g(n) is the number of moves so far and h(n) is the sum of Manhattan distances for tiles 1 through 8.

For each state, the solver records its best known path cost and parent state. When the goal is removed from the priority queue, the parent links are followed backward to reconstruct the complete path. The interface can then display the starting board, each intermediate board, the final board, and summary information.

3.1 SYSTEM ARCHITECTURE

INPUT LAYER	Initial 3 × 3 board and goal state
VALIDATION	Input checks and inversion-parity solvability test
STATE GENERATOR	Legal blank movements and successor states
A* SEARCH ENGINE	Priority queue; f(n) = g(n) + h(n)
HEURISTIC MODULE	Manhattan distance to the goal
PATH RECONSTRUCTION	Parent links, move sequence, and move count
OUTPUT / VISUALIZATION	Intermediate boards, solution status, and metrics

Figure 3.1: Architecture of the AI-Based 8-Puzzle Solver

3.2 ALGORITHM

• Read and validate the nine board values; require each of 0–8 exactly once.

• Count inversions among non-zero tiles. If the parity is incompatible with the goal, report that the puzzle is unsolvable.

• Initialize the A* priority queue with the initial state, path cost 0, and its Manhattan heuristic.

• Repeatedly remove the state with the lowest f value. If it is the goal, reconstruct and return the path.

• Generate legal successor states. If a successor has a lower known path cost, update its cost and parent and insert it into the queue.

• If the frontier becomes empty, report that no solution was found.

CHAPTER 4

MODULES DESCRIPTION

The proposed system is divided into six modules. Each module has a specific responsibility, making the implementation easier to test and maintain.

4.1 MODULE 1 – INPUT AND BOARD VALIDATION

This module accepts the initial board as nine numbers. It checks that the board has exactly nine entries and that every value from 0 to 8 appears once. The value 0 represents the blank space. Invalid entries are rejected with a clear message.

4.2 MODULE 2 – SOLVABILITY CHECKER

This module counts inversions by examining pairs of numbered tiles, ignoring the blank. An inversion occurs when a larger tile appears before a smaller tile. For the standard 3 × 3 goal state, an even inversion count indicates a solvable arrangement; an odd count indicates an unsolvable arrangement.

4.3 MODULE 3 – STATE-SPACE GENERATOR

The state generator identifies the blank's row and column and produces every legal adjacent swap. Boundary checks prevent moves outside the 3 × 3 board. Each resulting board is represented as an immutable tuple so that it can be stored in sets and dictionaries.

4.4 MODULE 4 – A* SEARCH AND HEURISTIC

A* search uses a priority queue to select the state with the smallest estimated total solution cost. The function f(n) = g(n) + h(n) combines the path cost g(n) and Manhattan-distance heuristic h(n). The heuristic is calculated by summing the row and column differences between each tile's current position and its goal position. The blank is excluded.

4.5 MODULE 5 – PATH RECONSTRUCTION

The algorithm stores a parent pointer and the move cost for discovered states. After the goal is found, the parent chain is followed from goal to start and reversed. This produces the ordered list of board states and allows the application to report the number of moves.

4.6 MODULE 6 – RESULT DISPLAY AND EVALUATION

This module displays whether a solution exists, the move-by-move board sequence, the final board, and metrics such as number of expanded states and elapsed execution time. These metrics support comparison with other search methods.

CHAPTER 5

IMPLEMENTATION AND RESULTS

The recommended implementation uses Python 3. A priority queue can be implemented with the heapq module, while dictionaries store path costs and parent links. The user interface may be a terminal interface or a simple graphical/web interface. The algorithm remains independent of the presentation layer.

5.1 EXPERIMENTAL SETUP

Item	Description
Language	Python 3
Core data structure	Tuple-based board states
Frontier	Priority queue (heapq)
Search method	A* search
Heuristic	Manhattan distance
Validation	Unique tile check and inversion parity
Evaluation metrics	Solution moves, expanded states, elapsed time

5.2 SAMPLE TEST CASES

Test	Initial board (0 = blank)	Expected behaviour
1	1 2 3 / 4 5 6 / 7 8 0	Already at goal; solution has 0 moves.
2	1 2 3 / 4 5 6 / 7 0 8	Solvable; one move to goal.
3	1 2 3 / 4 5 6 / 0 7 8	Solvable; two moves to goal.
4	1 2 3 / 4 5 6 / 8 7 0	Unsolvable for the standard goal; parity check rejects it.
5	Repeated or missing tile	Input validation reports an error.

5.3 RESULTS AND DISCUSSION

The test cases above provide functional checks for goal detection, one-step and two-step solutions, unsolvable-state rejection, and invalid-input handling. The expected behaviour is derived from the puzzle rules; it is not a claim that benchmark runs have already been performed.

For the final submission, run the implementation on the listed test cases and record actual elapsed time and expanded-state counts. A useful results table is shown below; enter measured values from your own execution instead of estimating them.

Test case	Solved?	Moves returned	Expanded states	Time (ms)
Already at goal	Yes	0	Fill after run	Fill after run
One move away	Yes	1	Fill after run	Fill after run
Two moves away	Yes	2	Fill after run	Fill after run

Odd inversion parity	No	—	Usually 0 search expansions	Fill after run

The A* algorithm with Manhattan distance is expected to find an optimal solution for standard equal-cost 8-puzzle moves because the heuristic is admissible and consistent. Its practical search effort depends on the starting configuration. The solver's explainable cost and heuristic values also make it useful for demonstrating why one state is expanded before another.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project describes an AI-Based 8-Puzzle Solver that applies classical search techniques to a well-defined problem. The puzzle is represented as a state space, legal tile movements are generated as operators, and solvability is checked before search. A* combines the cost already spent with Manhattan distance to prioritize states likely to lead to the goal. Parent tracking reconstructs a solution path that can be displayed to the user.

The project illustrates core Foundations of Artificial Intelligence concepts, including problem formulation, state-space search, heuristic functions, optimality, and performance measurement. The design is modular and can be implemented using standard Python libraries.

Future work can add a graphical interface with animated tile movements, allow users to enter or shuffle puzzles, compare BFS, DFS, Greedy Best-First Search, and A*, visualize the explored states, and generate performance charts over multiple test configurations. A further extension could generalize the solver to the 15-puzzle, while accounting for its larger search space.

CHAPTER 7

REFERENCES

1. Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson.

2. Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). “A Formal Basis for the Heuristic Determination of Minimum Cost Paths.” *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

3. Dijkstra, E. W. (1959). “A note on two problems in connexion with graphs.” *Numerische Mathematik*, 1, 269–271.

4. Python Software Foundation. *Python 3 Documentation*. Available at: https://docs.python.org/3/

5. Python Software Foundation. *heapq — Heap queue algorithm*. Available at: https://docs.python.org/3/library/heapq.html

6. Korf, R. E. (1985). “Depth-first iterative-deepening: An optimal admissible tree search.” *Artificial Intelligence*, 27(1), 97–109.

7. Nilsson, N. J. (1998). *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann.